Proyecto 2 - Celeste

Miguel Donis – 22993
Ian Farrington – 21952

14 de octubre de 2026

Diseño e implementación de un videojuego embebido
STM32F446RE, ESP32, pantalla ILI9341 y audio por DAC

Índice

1. Resumen **3**

2. Arquitectura del sistema **3**

3. Organización del programa principal **3**

4. Gráficos y administración de memoria **4**

4.1. Formato de texturas 4

4.2. Caché del nivel activo 4

5. Colisiones y lógica del escenario **4**

6. Movimiento de los jugadores **5**

7. Controles inalámbricos **6**

8. Sistema de sonido **6**

8.1. Circuito de salida 7

9. Puntaje y final de partida **7**

10. Esquemáticos **7**

10.1. Pantalla ILI9341 8

10.2. Tarjeta microSD 8

10.3. Controles inalámbricos 8

10.4. STM de música y salida de audio 9

10.5. Alimentación 9

11. Análisis de la implementación **9**

12. Uso de herramientas de inteligencia artificial **10**

12.1. Prompts utilizados 10

12.2. Ajuste de la física 10

12.3. Colisiones por píxel 11

13. Conclusiones **12**

14. links externos **12**

1. Resumen

El proyecto implementa un videojuego de plataformas para dos jugadores inspirado en *Celeste Classic*. La lógica principal se ejecuta en una STM32F446RE y se muestra en una pantalla ILI9341 de 240×320 píxeles. Dos ESP32 reciben los controles por Bluetooth y transmiten el estado de cada jugador a la STM principal. Una segunda STM32 genera la música mediante DAC, DMA y un temporizador.

Los niveles se almacenan como texto RGB565 en una tarjeta microSD. Al entrar a un nivel, la STM compone el fondo y la geometría, guarda el resultado en una región reservada de su Flash interna y crea un mapa compacto de colisiones en RAM. Esta distribución evita leer la SD durante el movimiento y permite restaurar únicamente la región ocupada por cada sprite.

2. Arquitectura del sistema

El sistema se dividió por función para que el procesamiento gráfico, los controles y el audio no compitan por el mismo periférico ni bloqueen el ciclo del juego.

Cuadro 1: Funciones de los componentes principales.

Componente	**Función**
STM32 principal	Ejecuta el juego, carga niveles, calcula física y colisiones, controla la ILI9341 y mantiene el puntaje.
ESP32 J1 y J2	Reciben cada control Bluetooth con Bluepad32 y envían tramas UART independientes.
STM32 de música	Recibe el estado del juego por SPI2 y genera las notas por DAC.
MicroSD	Almacena el fondo general, cinco niveles y tres imágenes finales.
ILI9341	Presenta el escenario, los sprites, el marcador y los resultados.
Amplificador	Adapta la señal analógica del DAC para manejar la bocina.

La pantalla utiliza un bus paralelo de 8 bits. La SD se conecta a SPI1, los ESP32 a UART5 y USART3, y la STM de música a SPI2. Esto evita compartir buses entre cargas con requisitos distintos.

3. Organización del programa principal

El programa usa un ciclo principal sin sistema operativo. Cada tarea revisa el tiempo con `HAL_GetTick()` y se ejecuta cuando corresponde. La física se actualiza cada 20 ms, equivalente a 50 actualizaciones por segundo. En cada vuelta se realizan las siguientes operaciones:

1. Actualizar la recepción de los dos controles.
2. Revisar eventos de inicio y el estado de la pantalla actual.
3. Actualizar la física de ambos jugadores cuando vence el periodo de 20 ms.
4. Procesar peligro, salida, puntaje y cambio de nivel.
5. Restaurar el fondo anterior y dibujar los sprites en su nueva posición.
6. Reenviar periódicamente a la STM de música el tema que debe reproducir.

El flujo general tiene cuatro estados visibles: menú, partida, resultados e imágenes finales. `START` inicia la partida; después del último nivel muestra el resultado y nuevas pulsaciones recorren las tres imágenes finales.

4. Gráficos y administración de memoria

4.1. Formato de texturas

Los colores se guardan en RGB565, por lo que cada píxel ocupa 16 bits. Los archivos de la SD conservan el formato hexadecimal original y pueden contener dos píxeles empaquetados en una palabra de 32 bits o valores RGB565 individuales. Una tabla de niveles registra el nombre, altura jugable, posición inicial, color transparente y formato de cada archivo. El número de niveles se calcula con `sizeof`, por lo que la lógica de progresión no contiene un límite escrito manualmente.

La composición se hace una fila a la vez:

$$C(x,y) = \begin{cases} B(x,y), & L(x,y) = T, \\ L(x,y), & L(x,y) \neq T, \end{cases} \tag{1}$$

donde B es el fondo general, L es la textura del nivel y T es su color transparente. Debajo del área jugable se genera una franja de marcador de color RGB565 `0x5E7B`.

4.2. Caché del nivel activo

Una imagen completa requiere

$$240 \cdot 320 \cdot 2 = 153\,600 \text{ bytes.} \tag{2}$$

La STM32F446RE tiene 128 KiB de RAM, por lo que el cuadro completo no cabe junto con el resto del programa. Se reservaron los sectores 6 y 7 de Flash, desde `0x08040000`, como una caché de 256 KiB. El enlazador limita el firmware a los primeros 256 KiB y evita que el código invada esa zona.

Durante la carga se leen el fondo y el nivel desde la SD, se compone cada fila, se genera su colisión y se escribe el resultado en Flash. Al finalizar se compara un hash FNV-1a de los datos producidos con el contenido almacenado. La SD queda fuera del ciclo de juego hasta el siguiente cambio de nivel.

Para mover un jugador no se redibuja toda la pantalla. Primero se recupera desde la Flash la región que ocupaba su sprite y después se dibuja el nuevo cuadro con transparencia. Esto reduce transferencia a la ILI9341 y evita rastros visibles.

5. Colisiones y lógica del escenario

El mapa de colisiones utiliza dos bits por píxel. Para los 76,800 píxeles ocupa 19,200 bytes de RAM y admite cuatro tipos:

Un píxel transparente es libre. Cualquier otro color de la textura es sólido, salvo los colores reservados para peligro y salida. Los límites de la pantalla también se consideran sólidos.

Cuadro 2: Tipos guardados en el mapa de colisiones.

Valor	Tipo	Comportamiento
0	Libre	El jugador puede ocupar el píxel.
1	Sólido	Suelo, pared, techo y franja del marcador.
2	Peligro	Rojo `0xF800`; reinicia la posición del jugador.
3	Salida	Amarillo `0xFF60` o `0xFFE0`; completa el nivel.

La caja de colisión del jugador mide 18×14 píxeles. Para verificar una posición, el código recorre todo el rectángulo y consulta el mapa. El movimiento se resuelve primero en el eje horizontal y luego en el vertical, avanzando un píxel por iteración. Si el siguiente paso colisiona, la posición queda en el último punto válido y la velocidad de ese eje se anula. Este recorrido evita atravesar superficies cuando una velocidad representa varios píxeles por actualización.

El suelo no se almacena como un tipo diferente de la pared. Se reconoce probando la misma caja un píxel debajo del jugador. Las paredes se buscan a un píxel de los laterales y el techo se detecta cuando el movimiento vertical ascendente encuentra una región sólida.

6. Movimiento de los jugadores

La posición y velocidad usan punto fijo con 256 unidades internas por píxel. Esto permite aceleraciones suaves sin depender de números flotantes durante cada cuadro. Los multiplicadores decimales se aplican al compilar para producir constantes enteras redondeadas.

Cada jugador conserva su posición, velocidad, orientación, cuadro de animación, contador de saltos y estados temporales. La velocidad horizontal se aproxima de forma gradual a una velocidad objetivo. Se usan aceleraciones separadas para suelo y aire, además de desaceleración cuando se libera la dirección.

El contador de saltos tiene tres estados: salto normal disponible, segundo salto disponible y saltos agotados. El salto normal alcanza actualmente 2.5 veces la altura del jugador y habilita el segundo. El segundo salto alcanza tres veces la altura y depende de la dirección mantenida:

- Sin dirección, o solamente arriba: impulso vertical.
- Dirección lateral sin arriba: desplazamiento horizontal con distancia limitada; la gravedad se pausa hasta alcanzar el límite o chocar.
- Arriba y una dirección lateral: impulso diagonal.

Al aterrizar se restablece el contador. El salto de pared expulsa al jugador hacia el lado opuesto y aplica un bloqueo corto de dirección para conservar el impulso. El contacto lateral durante una caída limita la velocidad al 55 % de la velocidad terminal normal. Un salto de pared no recupera un segundo salto ya consumido.

También se mantiene un búfer de cuatro cuadros para el salto normal. Una pulsación recibida ligeramente antes de cumplir la condición puede ejecutarse en las actualizaciones siguientes, lo que hace el control menos dependiente del instante exacto de recepción.

7. Controles inalámbricos

Cada ESP32 ejecuta Bluepad32 y acepta únicamente la dirección Bluetooth registrada para su jugador. El estado se convierte en una máscara de siete bits: izquierda, derecha, arriba, abajo, Cruz, Círculo y Start. Las combinaciones se conservan porque cada bit se procesa de manera independiente.

El ESP32 transmite por UART a 115200 bit/s una trama cada 20 ms:

```
[0xA5, ID, botones, estado, CRC-8]
```

El byte de estado es `0xA1` cuando el control está conectado y `0xA0` cuando está desconectado. El CRC-8 usa polinomio `0x07` y valor inicial `0x00`. La STM reconstruye las tramas mediante interrupciones de un byte, valida sincronización, ID, estado y CRC, y mantiene una cola independiente para cada jugador.

Se distinguen botones mantenidos de eventos de pulsación. Las direcciones se leen como estado continuo; salto, segundo salto y Start generan un evento únicamente al pasar de liberado a presionado. Si no llega una trama válida durante 300 ms, las acciones mantenidas se liberan para evitar que el jugador continúe moviéndose por una desconexión.

Cuadro 3: Asignación de controles.

Entrada	**Acción**
Cruceta izquierda/derecha	Movimiento horizontal
Cruceta arriba	Dirección del segundo salto
Cruceta abajo	Disponible en el protocolo
Cruz	Salto normal o salto de pared
Círculo	Segundo salto direccional
Start	Iniciar partida o avanzar pantallas finales

8. Sistema de sonido

La STM principal envía un byte por SPI2: `'M'` para menú y resultados, o `'T'` para la partida. El comando se manda al cambiar de estado y se repite cada segundo para recuperar la sincronización si una de las placas se reinicia. La STM de música funciona como esclavo SPI en modo 0, con datos de 8 bits y bit más significativo primero.

Las canciones se almacenan como dos arreglos paralelos: frecuencia y duración de cada nota. Una máquina de estados basada en `HAL_GetTick()` avanza la canción sin usar `HAL_Delay()`, por lo que la recepción SPI permanece activa. Un comando repetido no reinicia la melodía.

El audio se genera con una tabla senoidal de 128 muestras. TIM6 produce el disparo del DAC y su periodo se calcula para cada frecuencia f:

$$ARR \approx \frac{84\,000\,000}{128 f} - 1. \tag{3}$$

DMA transfiere la tabla al DAC1 en modo circular. Una frecuencia de cero detiene el temporizador y

coloca el DAC en 2048, el punto medio de 12 bits. El volumen digital se ajusta al 50 % escalando la amplitud respecto a ese punto medio.

8.1. Circuito de salida

La salida `PA4/DAC1_OUT1` se conecta a un capacitor electrolítico de 10 μF, con el terminal positivo hacia la STM. El otro terminal entra a un canal del amplificador. El capacitor elimina el nivel DC de la señal. La bocina se conecta entre las dos terminales de salida del mismo canal del amplificador; ninguna terminal de la bocina se conecta a tierra.

```
PA4 DAC → 10 uF → entrada L del amplificador → L+/L- → bocina
```

El amplificador trabaja con 5 V y comparte tierra con ambas STM y los ESP32. Se usa una fuente común regulada para evitar que los picos de consumo del amplificador o de los radios Bluetooth afecten la lógica digital.

9. Puntaje y final de partida

Cada nivel muestra una franja inferior con el identificador y puntaje de ambos jugadores. Cuando alguien toca la salida amarilla, el primer evento procesado recibe un punto, ambos jugadores se reinician y se carga el nivel siguiente. Los puntajes se mantienen en variables de 16 bits durante toda la partida.

Después del quinto nivel se compara el total y se muestra ganador o empate sobre la pantalla de inicio. La música cambia al tema de menú. A continuación, cada pulsación de Start carga desde la SD una de las tres imágenes finales. Al terminar la secuencia se regresa al menú y una partida nueva reinicia ambos puntajes.

10. Esquemáticos

Esta sección resume las conexiones físicas empleadas en el proyecto. Las señales lógicas trabajan a 3.3 V y todos los módulos comparten la misma referencia de tierra. Los diagramas esquemáticos se agregarán junto a estas tablas.

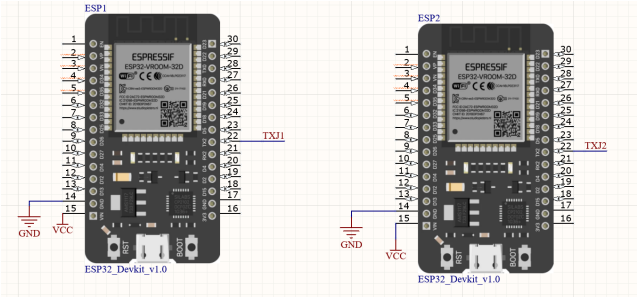


Figura 1: Imagen 1. - Esquematicos ESP controles

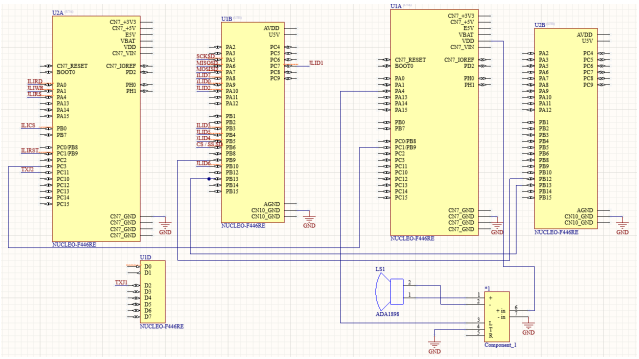


Figura 2: imagen 2. esquemático stm - juego y audio

10.1. Pantalla ILI9341

La pantalla utiliza una interfaz paralela de 8 bits. Las líneas de datos y control se conectan a la STM principal de la siguiente manera:

Cuadro 4: Conexiones entre la STM principal y la pantalla ILI9341.

Señal ILI9341	**Pin STM principal**
D0	PA9
D1	PC7
D2	PA10
D3	PB3
D4	PB5
D5	PB4
D6	PB10
D7	PA8
RD	PA0
WR	PA1
RS/DC	PA4
RST	PC1
CS	PB0
VCC y GND	3.3 V y tierra común

10.2. Tarjeta microSD

La tarjeta se comunica mediante SPI1 y se utiliza únicamente durante la carga de un nivel o de una imagen final.

10.3. Controles inalámbricos

Cada ESP32 envía por UART únicamente la trama correspondiente a su jugador. No se utiliza la línea de recepción de los ESP32.

Cuadro 5: Conexiones del módulo microSD.

Señal microSD	**Pin STM principal**
SCK	PA5 / SPI1_SCK
MISO	PA6 / SPI1_MISO
MOSI	PA7 / SPI1_MOSI
CS/SS	PB6
VCC y GND	3.3 V y tierra común

Cuadro 6: Conexiones de los ESP32 con la STM principal.

Módulo	**STM principal**	**Función**
ESP32 J1 GPIO17/TX2	PD2/UART5_RX	Recepción del control del jugador 1
ESP32 J2 GPIO17/TX2	PC11/USART3_RX	Recepción del control del jugador 2
GND de ambos ESP32	GND común	Referencia de las señales UART

10.4. STM de música y salida de audio

La STM principal actúa como maestro de SPI2 y envía un byte para seleccionar el tema. La STM de música funciona como esclavo y solamente recibe, por lo que no se requiere una línea MISO.

Cuadro 7: Comunicación entre la STM principal y la STM de música.

STM principal	**STM de música**	**Señal**
PC3/SPI2_MOSI	PC1/SPI2_MOSI	Comando de selección de tema
PB13/SPI2_SCK	PB13/SPI2_SCK	Reloj SPI
PB9/SPI2_NSS	PB12/SPI2_NSS	Selección del esclavo
GND	GND	Tierra común

La salida `PA4/DAC1_OUT1` de la STM de música se conecta al terminal positivo de un capacitor electrolítico de 10 μF. El terminal negativo entra al canal izquierdo del amplificador. La bocina se conecta entre `L+` y `L−`; ninguna de sus terminales se conecta directamente a tierra.

10.5. Alimentación

La alimentación se distribuye en paralelo desde una fuente regulada de 5 V. Las dos placas Nucleo reciben esta tensión por `E5V`, los ESP32 por `VIN/5V` y el amplificador por su entrada de 5 V. La pantalla y la microSD utilizan 3.3 V. Todas las tierras se conectan entre sí.

11. Análisis de la implementación

La principal restricción fue la memoria. Leer directamente desde la SD durante el juego producía pausas y hacía que la restauración del personaje dependiera del tiempo de acceso del sistema de archivos. La caché en Flash resolvió este problema sin reservar un cuadro imposible de alojar en RAM. Como costo, cada cambio de nivel requiere borrar sectores internos y esperar la carga completa. El desgaste se limita porque la Flash no se reescribe al morir, únicamente al cambiar de nivel.

El mapa de dos bits por píxel permite consultar colisiones en tiempo constante y distinguir geometría,

peligro y salida con un uso moderado de RAM. Separar posición subpíxel y colisión por píxel mantiene movimiento suave sin complicar la geometría.

La división del sonido en otra STM evita que la temporización del DAC y el acceso DMA afecten la pantalla o la física. Del mismo modo, cada ESP32 tiene su propia UART, por lo que los dos jugadores no comparten una línea ni requieren arbitraje.

Entre las limitaciones actuales, una llegada simultánea a la salida favorece al primer jugador procesado, las texturas en texto requieren más tiempo de carga que un formato binario y la comunicación de música no incluye confirmación de recepción. Estas decisiones son aceptables para la escala actual y mantienen el código legible.

12. Uso de herramientas de inteligencia artificial

Durante el desarrollo se utilizó **ChatGPT-5.6 Sol** como herramienta de apoyo para analizar problemas relacionados con la física, las colisiones y la organización del código. También se utilizó para revisar la redacción de este informe. Las decisiones de diseño, la integración del hardware, la adaptación del código y las pruebas fueron realizadas y verificadas por los autores.

12.1. Prompts utilizados

Algunas de las consultas realizadas durante el proyecto fueron las siguientes:

1. "Ayúdame a ajustar la gravedad, la altura de los saltos y el movimiento horizontal para que el personaje se mueva de forma fluida."
2. "Revisa la lógica de colisiones por píxel y explica cómo evitar que el jugador atraviese paredes, suelo o techo."
3. "Revisa el informe realizado y adáptalo evitando faltas de ortografía y redundancias para tener un resultado más profesional."
4. "En base a los comentarios realizados hasta el momento, completa los comentarios del código manteniendo buenas prácticas para que sea fácil de entender."

12.2. Ajuste de la física

El siguiente fragmento resume el uso de punto fijo y multiplicadores para ajustar el movimiento sin ejecutar operaciones de punto flotante en cada cuadro. Este bloque se desarrolló con apoyo de IA y después se calibró mediante pruebas en el juego.

Listing 1: Parámetros principales de movimiento ajustados con apoyo de IA.

```
#define GAME_FIXED_POINT_ONE 256L
#define GAME_JUMP_HEIGHT_MULTIPLIER 2.5f
#define GAME_SECOND_JUMP_HEIGHT_MULTIPLIER 3.0f
#define GAME_GRAVITY_MULTIPLIER 2.0f

#define GAME_SCALE_ROUNDED(base, multiplier) \
  ((int32_t)(((float)(base) * (multiplier)) + 0.5f))
```

```

gravity = (velocity_y >= -GAME_APEX_SPEED &&
           velocity_y <= GAME_APEX_SPEED)
              ? GAME_APEX_GRAVITY
              : GAME_GRAVITY;
velocity_y = Game_Approach(velocity_y,
                           GAME_MAX_FALL_SPEED,
                           gravity);
```

La solución separa la gravedad normal de la gravedad cercana al punto más alto del salto. Los multiplicadores permiten modificar alturas y aceleración con valores decimales, mientras que el resultado final se convierte a las unidades enteras del sistema de punto fijo. Esto permitió ajustar el movimiento sin introducir cambios bruscos entre cuadros.

12.3. Colisiones por píxel

Para impedir que una velocidad alta atraviese una superficie, se revisa el rectángulo completo del jugador y el desplazamiento se aplica un píxel a la vez. El siguiente bloque también fue desarrollado con apoyo de IA y adaptado al mapa de colisiones del proyecto.

Listing 2: Comprobación de la caja completa del jugador.

```
static uint8_t Game_CollidesAt(int32_t x, int32_t y)
{
  for (uint16_t row = 0; row < GAME_PLAYER_HEIGHT; row++)
  {
    for (uint16_t column = 0; column < GAME_PLAYER_WIDTH; column++)
    {
      if (Game_IsSolidPixel(x + column, y + row))
      {
        return 1U;
      }
    }
  }
  return 0U;
}

while (current_x != target_x)
{
  int32_t next_x = current_x + ((target_x > current_x) ? 1L : -1L);
  if (Game_CollidesAt(next_x, player_y))
  {
    velocity_x = 0L;
    break;
  }
  current_x = next_x;
}
```

La caja completa evita detectar únicamente una esquina del sprite. El recorrido intermedio garantiza que cada posición sea validada antes de avanzar. El mismo criterio se aplica en el eje vertical, por lo que suelo, techo y paredes se resuelven con una sola función de colisión.

13. Conclusiones

El proyecto integra gráficos, almacenamiento, comunicación inalámbrica, física y audio en una plataforma con recursos limitados. La separación entre imagen compuesta en Flash y mapa de colisiones en RAM permitió mantener una actualización estable sin consultar la SD durante la partida. La lógica por estados y las tareas basadas en tiempo evitan bloqueos entre controles, animación y música.

La implementación queda preparada para agregar niveles mediante su tabla de configuración y para ajustar el movimiento mediante multiplicadores definidos en el código. La arquitectura de dos controles y dos procesadores STM también mantiene separadas las funciones de juego y reproducción de audio.

14. links externos

Video del funcionamiento: https://youtu.be/-hGdZbvIJHU

El video completo, con la explicación del código y la demostración del funcionamiento, está incluido en el archivo ZIP del proyecto como `Video_Celeste_final.mp4`.

Repositorio ode github: https://github.com/DonMiguelMatta/Digital-2/tree/main/STM/Proyecto2